

DESIGN AND ANALYSIS OF ALGORITHMS

MODULE 1 — Introduction to Algorithms & Performance Analysis

1. Introduction to Algorithms

An **algorithm** is a step-by-step method used to solve a problem.

It tells us what to do, in what order, and when to stop.

Before writing a program, we can first design the algorithm.

Real-Time Example — ATM

```
START
↓
Insert Card
↓
Enter PIN
↓
Select Withdrawal
↓
Enter Amount
↓
Check Balance
↓
Dispense Money
↓
STOP
```

All these steps together form a procedure for withdrawing money.

Example

Problem: Add two numbers.

```
1. Start
2. Read A and B
3. Sum = A + B
4. Display Sum
5. Stop
```

Remember

Algorithm = Step-by-step solution to a problem.

2. Characteristics of an Algorithm

A good algorithm should have **input, output, clear steps, finite steps, and effective operations**.

Every step should be easy to understand and the algorithm must eventually stop.

These properties make an algorithm reliable and useful.

Main Characteristics

Characteristic	Meaning
--	
Input	Data given to the algorithm
Output	Result produced
Definiteness	Steps must be clear
Finiteness	Must eventually stop
Effectiveness	Steps must be practical

Simple Flow

```
graph TD; Input --> Process; Process --> Output; Output --> Stop;
```

3. Role and Importance of Algorithms

Algorithms help us solve problems in a **systematic and efficient way**.

They help reduce unnecessary work and make programs faster and easier to design.

Their importance increases when the amount of data becomes very large.

Real-Time Example

A college may have **10,000 student records**.

Searching each record one by one can take more work.

A better search algorithm can find the required record much faster.

Applications

- Google Search

- GPS Navigation
- Online Shopping
- Banking Systems
- Social Media
- Artificial Intelligence

Good algorithm → Better performance

4. Algorithm Design Techniques

Algorithm design techniques are **different methods used to solve different types of problems**.

A problem may be solved using more than one technique, but one technique may be more efficient.

Important techniques include Divide and Conquer, Greedy, Dynamic Programming, Backtracking and Branch and Bound.



5. Divide and Conquer

Divide and Conquer means **breaking a large problem into smaller problems**.

The smaller problems are solved and their results are combined.

The three basic steps are **Divide → Conquer → Combine**.

Diagram



Real-Time Example

Suppose 100 files have to be processed.

```
100 Files
  ↓
50 + 50
  ↓
25 + 25 + 25 + 25
```

The large problem becomes easier by dividing it.

Examples

- Binary Search
- Merge Sort
- Quick Sort

6. Greedy Method

The **Greedy Method** chooses the best available option at the current step.

It tries to make a good local choice and continues toward the final answer.

It is commonly used in optimization problems.

Real-Time Example

Suppose you are selecting the shortest available road at each stage of a journey.

```
START
  ↓
Best Choice
  ↓
Best Choice
  ↓
Best Choice
  ↓
DESTINATION
```

Examples

- Prim's Algorithm
- Kruskal's Algorithm
- Fractional Knapsack
- Dijkstra's Algorithm

Greedy = Choose the best available option now.

7. Dynamic Programming

Dynamic Programming solves smaller problems and **stores their answers**.

When the same smaller problem occurs again, the stored answer is reused.

This avoids repeated calculations and saves time.

Basic Idea

```
Solve
↓
Store
↓
Problem appears again?
↓
Use stored answer
```

Real-Time Example

A system calculates a student's total once and stores it.

When the total is needed again, it uses the saved result instead of recalculating it.

Examples

- Fibonacci
- 0/1 Knapsack
- LCS
- Matrix Chain Multiplication

Dynamic Programming = Solve → Store → Reuse

8. Backtracking

Backtracking tries one possible solution at a time.

If the choice fails, it **goes back** and tries another choice.

It is useful when there are many possible combinations.

Real-Time Example — Maze

```
START
↓
Choose Path A
↓
Dead End
↓
GO BACK
↓
Choose Path B
↓
DESTINATION
```

Examples

- N-Queen
- Sudoku
- Maze Problem
- Graph Coloring

Try → Check → Go Back → Try Again

9. Branch and Bound

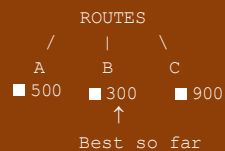
Branch and Bound divides a problem into different possible choices.

It checks whether a choice can still give a better answer.

Choices that cannot improve the solution are discarded.

Real-Time Example

Suppose we want the cheapest route:



A route that cannot beat the current best solution need not be explored further.

Examples

- Travelling Salesman Problem
- 0/1 Knapsack
- Job Assignment

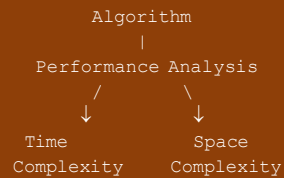
Branch = Explore choices; Bound = Remove poor choices.

10. Performance Analysis

Performance analysis means checking **how efficiently an algorithm works**.

We mainly study the time required and the memory required.

This helps us compare two algorithms solving the same problem.



Example

```
Algorithm A → 10,000 operations  
Algorithm B → 1,000 operations
```

For the same task, Algorithm B is generally more efficient in terms of work.

11. Time Complexity

Time Complexity tells us **how the amount of work grows when the input size grows**.

It does not necessarily mean the exact time in seconds.

We usually count basic operations.

C Example

```
#include <stdio.h>

int main()
{
    int n = 5;

    for(int i = 0; i < n; i++)
    {
        printf("%d ", i);
    }

    return 0;
}
```

The loop runs n times.

Therefore:

$O(n)$

Simple Example

```
10 items → 10 operations  
100 items → 100 operations  
1000 items → 1000 operations
```

12. Space Complexity

Space Complexity tells us **how much extra memory an algorithm needs**.

Memory may be required for variables, arrays, recursion, and other data structures.

When the input becomes larger, the memory requirement may also increase.

C Example

```
int marks[n];
```

The array size depends on n .

Therefore:

$O(n)$

Remember

Time → Work

Space → Memory

13. Time Complexity vs Space Complexity

Time Complexity measures the **amount of computational work**.

Space Complexity measures the **extra memory required**.

An algorithm should ideally be efficient in both.

Time Complexity	Space Complexity
Measures computational work	Measures memory
Related to speed	Related to storage
Example: $O(n)$	Example: $O(n)$

Real-Time Example

One program may be very fast but use a lot of memory, while another uses less memory but takes more time.

This is called a **time-space trade-off**.

14. Best Case, Average Case and Worst Case

The same algorithm may behave differently for different inputs.

Best Case means minimum work, **Average Case** means expected work, and **Worst Case** means maximum work.

Linear Search is the easiest example.

Linear Search Example

10 20 30 40 50

Best Case

Search 10:

10 ← Found

Only one comparison.

$O(1)$

Average Case

Search for an element somewhere in the middle.

Approximately:

$O(n)$

Worst Case

Search 50:

10 → 20 → 30 → 40 → 50

All elements may be checked.

$O(n)$

Summary

Best Case → Minimum work
Average Case → Expected work
Worst Case → Maximum work

Asymptotic Analysis, Growth of Functions & Recurrence Relations

15. Asymptotic Notations

Asymptotic notation is used to describe **how an algorithm behaves when the input size becomes very large.**

It helps us compare algorithms without depending on the computer's speed or programming language.

The three important notations are **Big O**, **Big Ω** and **Big Θ** .

Main Notations



16. Big O Notation — O

Big O represents the **upper bound** of an algorithm's growth.

It tells us how much the running time can grow as the input size becomes large.

It is commonly used to express the **worst-case growth**.

Real-Time Example

Suppose you are searching for a name in an unsorted list.



If the required name is last, all elements may need to be checked.

Therefore:

$O(n)$

Simple Meaning

Big O = At most this much work.

17. Big Omega — Ω

Big Omega represents the **lower bound** of an algorithm's growth.

It tells us the minimum amount of work represented by the bound.

For example, Linear Search can find an element in the first position.

Example

```
10 20 30 40 50
↑
Found
```

Only one comparison is needed.

$\Omega(1)$

Simple Meaning

Big Ω = At least this much work.

18. Big Theta — Θ

Big Theta gives a **tight bound** when the upper and lower bounds grow at the same rate.

It gives the overall asymptotic growth of an algorithm more precisely than giving only an upper or lower bound.

For example, if an algorithm always processes all n elements once, its growth is $\Theta(n)$.

Real-Time Example

Taking attendance by calling **every student**:

```
Student 1
Student 2
Student 3
...
Student n
```

You must check every student.

Therefore:

$\Theta(n)$

Simple Meaning

Big Θ = The actual growth is tightly bounded by this rate.

19. Comparison of O , Ω and Θ

Notation	Simple Meaning	Idea
O	Upper bound	At most this growth
Ω	Lower bound	At least this growth
Θ	Tight bound	Same growth rate

Easy Memory

O → Upper
 Ω → Lower
 Θ → Tight

Important Note

Best/Worst/Average cases and asymptotic notations are **different ideas**.

- Best/Worst/Average → describe different input situations.
- $O/\Omega/\Theta$ → describe mathematical bounds on growth.

20. Common Growth Rates

Growth rate shows **how quickly the work increases when n increases**.

Complexity	Name	Example
$O(1)$	Constant	Array access
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort
$O(2^{\blacksquare})$	Exponential	Some recursive problems
$O(n!)$	Factorial	Brute-force permutations

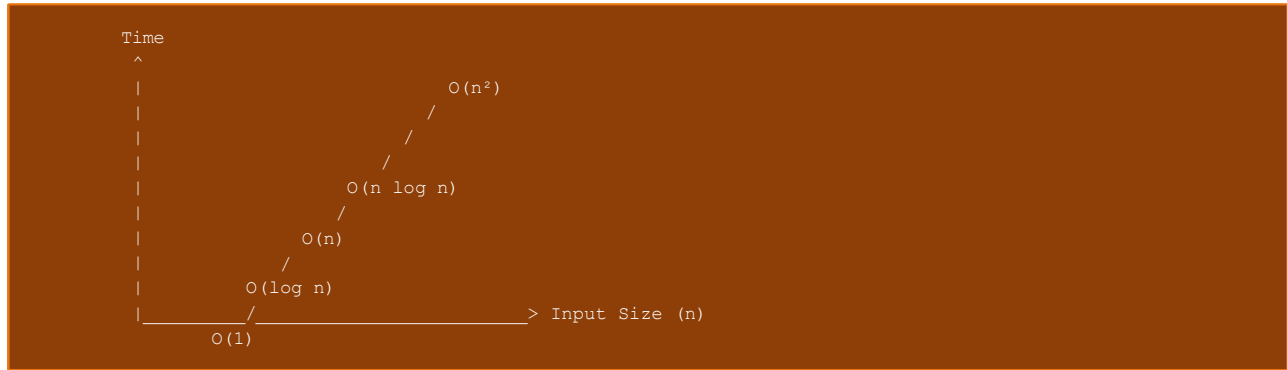
Growth Order

$O(1)$
↓
 $O(\log n)$
↓
 $O(n)$
↓
 $O(n \log n)$
↓
 $O(n^2)$
↓
 $O(2^{\blacksquare})$
↓
 $O(n!)$

For large inputs, a slower-growing function is generally preferable.

21. Graph of Common Growth Rates

This graph helps students see why complexity matters.



How to understand the graph

- $O(1)$ → almost no increase
- $O(\log n)$ → very slow increase
- $O(n)$ → direct increase
- $O(n \log n)$ → faster than linear
- $O(n^2)$ → very fast increase

22. Growth of Functions

Growth of functions tells us **how quickly the work of an algorithm increases when the input becomes larger.**

Two algorithms may solve the same problem, but one may become much slower when n becomes large.

This is why we compare their growth rates.

Searching Example

Linear Search

$n \rightarrow n$

$O(n)$

Binary Search

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

$O(\log n)$

Sorting Example

Bubble Sort

$O(n^2)$

Merge Sort

$O(n \log n)$

Simple Conclusion

Growth of functions tells us how an algorithm behaves when the amount of data becomes large.

23. Recurrence Relations

A recurrence relation describes the running time of a **recursive algorithm** using a **smaller version of the same problem**.

It is mainly used to analyze recursive and Divide-and-Conquer algorithms.

The current problem is expressed in terms of one or more smaller problems.

Simple Example

Suppose a function calls itself with $n-1$.

```
fun (5)
  ↓
fun (4)
  ↓
fun (3)
  ↓
fun (2)
  ↓
fun (1)
```

We can represent its work as:

$$T(n) = T(n-1) + 1$$

Meaning

- $T(n) \rightarrow$ work for current problem
- $T(n-1) \rightarrow$ work for smaller problem
- $+1 \rightarrow$ current extra work

24. Why Do We Need Recurrence Relations?

Normal loops are often easy to analyze directly.

Recursive algorithms are harder because a function **calls itself again and again**.

Recurrence relations give us a mathematical way to find their time complexity.

Examples

- Factorial
- Binary Search
- Merge Sort
- Quick Sort

Basic Idea

```
Recursive Algorithm
↓
Smaller Problem
↓
Recurrence Relation
↓
Solve It
↓
Time Complexity
```

25. Substitution Method

The Substitution Method solves a recurrence by **repeatedly replacing the smaller recurrence**.

We continue until we reach the base case.

Then we identify the total number of operations.

Example

$$T(n) = T(n-1) + 1$$

Substitute:

$$T(n) = T(n-2) + 1 + 1$$

Again:

$$T(n) = T(n-3) + 1 + 1 + 1$$

Continue until:

$$T(1)$$

So there are approximately n steps.

$$T(n) = O(n)$$

Easy Memory

Substitution = Keep replacing.

26. Example — Binary Search Using Substitution

Binary Search reduces the problem to half.

$$T(n) = T(n/2) + 1$$

Substitute:

$$T(n) = T(n/4) + 1 + 1$$

Again:

$$T(n) = T(n/8) + 1 + 1 + 1$$

Continue:

$$T(n) = T(n/2^k) + k$$

When the problem becomes 1:

$$\frac{n}{2^k} = 1$$

Therefore:

$$k = \log_2 n$$

So:

$$T(n) = O(\log n)$$

Simple Idea

The problem is divided by 2 again and again, so the number of steps grows logarithmically.

27. Recursion Tree Method

The Recursion Tree Method represents recursive calls in the form of a **tree**.

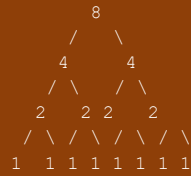
We calculate how much work is done at each level and how many levels exist.

Then we add the work from all levels.

Example — Merge Sort

$$T(n) = 2T(n/2) + n$$

For 8 elements:



Work at each level

```
Level 0 → 8
Level 1 → 4 + 4 = 8
Level 2 → 2 + 2 + 2 + 2 = 8
Level 3 → 1 + 1 + ... + 1 = 8
```

So work at each level is approximately:

n

Number of levels is approximately:

$\log n$

Therefore:

Total Work = $n \log n$

$O(n \log n)$

Easy Memory

Recursion Tree = Draw → Find work at each level → Find number of levels → Add.

28. Master Theorem

Master Theorem is a **shortcut for finding the time complexity of many Divide-and-Conquer algorithms**.

Instead of drawing the full recursion tree every time, we can use the theorem.

It is commonly used for recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

Meaning of the Parts

Part	Meaning
a	Number of smaller problems

Part	Meaning
n/b	Size of each smaller problem
$f(n)$	Extra work

29. Master Theorem — Simple Example

Merge Sort:

$$T(n) = 2T(n/2) + n$$

Here:

2 → two smaller problems
 $n/2$ → each is half the size
 $+n$ → merging/extra work

Master Theorem gives:

$$O(n \log n)$$

Simple Meaning

Master Theorem is a shortcut to find the complexity of many Divide-and-Conquer algorithms.

30. Three Cases of Master Theorem

For BCA students, first remember the basic idea:

Case 1

The recursive part grows faster than the extra work.

Recursive part dominates.

Case 2

Both parts have approximately the same growth.

An extra $\log n$ factor appears.

Example:

$$T(n) = 2T(n/2) + n$$

Result:

$O(n \log n)$

Case 3

The extra work grows faster than the recursive part.

Extra work dominates.

Easy Memory

```
Case 1 → Recursive part bigger  
Case 2 → Both same  
Case 3 → Extra work bigger
```

C PROGRAMS

Recursive Factorial

This is useful for understanding **recursion and recurrence relations**.

```
#include <stdio.h>

int factorial(int n)
{
    if(n == 0)
        return 1;

    return n * factorial(n - 1);
}

int main()
{
    int n = 5;

    printf("Factorial = %d", factorial(n));

    return 0;
}
```

Working

```
factorial(5)
  ↓
factorial(4)
  ↓
factorial(3)
  ↓
factorial(2)
  ↓
factorial(1)
```

Time Complexity:

```
[  
O(n)  
]
```

Iterative vs Recursive Factorial

Iterative

Uses a loop.

```
n → n-1 → n-2 → ... → 1
```

Recursive

Function calls itself.

```
factorial(n)  
  ↓  
factorial(n-1)  
  ↓  
factorial(n-2)
```

Both have:

```
[  
O(n)  
]
```

time complexity, but recursive factorial uses additional **call-stack memory**.

Linear Search — Case Analysis

```
#include <stdio.h>

int main()
{
    int a[] = {10, 20, 30, 40, 50};
    int n = 5;
    int key = 30;

    for(int i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            printf("Element found");
            return 0;
        }
    }

    printf("Element not found");

    return 0;
}
```

Complexity

- Best Case $\rightarrow O(1)$
- Average Case $\rightarrow O(n)$
- Worst Case $\rightarrow O(n)$

Binary Search — Growth Example

Binary Search works only with **sorted data**.

```
10 20 30 40 50 60 70
      ↑
    Middle
```

The search area becomes:

```
n
↓
n/2
↓
n/4
↓
n/8
↓
...
```

Therefore:

$O(\log n)$

1. Bubble Sort

Bubble Sort compares **adjacent elements** and swaps them when they are in the wrong order.

After each pass, larger elements move toward the end.

It is simple but becomes slow for large input.

Example

```
5 3 4 1
↓ ↓
3 5 4 1
  ↓ ↓
  3 4 5 1
```

C Program

```
#include <stdio.h>

int main()
{
    int a[] = {5, 3, 4, 1, 2};
    int n = 5;
    int i, j, temp;

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array: ");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Complexity

Worst Case: $O(n^2)$

2. Insertion Sort

Insertion Sort takes one element at a time and places it in its **correct position** among the previous elements.

It works like arranging playing cards in your hand.

It performs well when the data is already or nearly sorted.

Example

```
7 3 5 2
↓
3 7
↓
3 5 7
↓
2 3 5 7
```

C Program

```
#include <stdio.h>

int main()
{
    int a[] = {7, 3, 5, 2, 4};
    int n = 5;
    int i, j, key;

    for(i = 1; i < n; i++)
    {
        key = a[i];
        j = i - 1;

        while(j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }

    printf("Sorted array: ");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Complexity

Worst Case: $O(n^2)$

Best Case: $O(n)$ for already sorted input.

3. Selection Sort

Selection Sort repeatedly finds the **smallest element** from the unsorted part.

It then places that element in its correct position.

The process continues until the complete array is sorted.

Example

```
64  25  12  22  11
↓
Find minimum = 11

11  25  12  22  64
```

Then find the next minimum and continue.

C Program

```
#include <stdio.h>

int main()
{
    int a[] = {64, 25, 12, 22, 11};
    int n = 5;
    int i, j, min, temp;

    for(i = 0; i < n - 1; i++)
    {
        min = i;

        for(j = i + 1; j < n; j++)
        {
            if(a[j] < a[min])
                min = j;
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("Sorted array: ");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Complexity

Best, Average and Worst:

$O(n^2)$

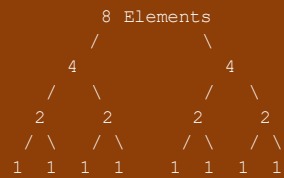
4. Merge Sort

Merge Sort uses the **Divide and Conquer** technique.

It divides the array into smaller parts, sorts those parts, and then merges them.

It is much more efficient than simple quadratic sorting methods for large data.

Diagram



Then the elements are merged back in sorted order.

C Program

```

#include <stdio.h>

void merge(int a[], int low, int mid, int high)
{
    int i = low, j = mid + 1, k = 0; int
    temp[100];

    while(i <= mid && j <= high)
    {
        if(a[i] < a[j]) temp[k++] =
            a[i++];
        else
            temp[k++] = a[j++];
    }

    while(i <= mid) temp[k++] =
        a[i++];

    while(j <= high) temp[k++] =
        a[j++];

    for(i = low, k = 0; i <= high; i++, k++) a[i] =
        temp[k];
}

void mergeSort(int a[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

int main()
{
    int a[] = {38, 27, 43, 3, 9, 82, 10, 5};
    int n = 8; int
    i;

    mergeSort(a, 0, n - 1);

    printf("Sorted array: "); for(i
    = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Complexity

$O(n \log n)$

IMPORTANT QUESTIONS

2 Marks – 15 Questions

1. Define an algorithm.
2. What is a recursive algorithm?
3. What is time complexity?
4. What is space complexity?
5. What is asymptotic notation?
6. Write any two characteristics of an algorithm.
7. What is a basic operation in an algorithm?
8. What is performance analysis?
9. What is best-case efficiency?
10. What is average-case efficiency?
11. What is worst-case efficiency?
12. Define Big O notation.
13. Define Big Ω notation.
14. Define Big Θ notation.
15. What is a recurrence relation?

5 Marks – 10 Questions

1. Write a note on asymptotic notations.
2. Briefly explain best-case, average-case and worst-case efficiency with an example.
3. Explain the general plan of mathematical analysis of a recursive algorithm with an example.
4. Give the general plan of mathematical analysis of a non-recursive algorithm with an example.
5. Write a note on units for measuring running time.
6. Explain the characteristics of an algorithm.
7. Explain time complexity and space complexity with suitable examples.
8. Explain growth of functions with suitable examples.
9. Explain recurrence relations with an example.
10. Explain the substitution method and recursion tree method with examples.

10 Marks – 6 Questions

1. Explain asymptotic notations with their efficiency and types.
2. Explain the fundamentals of algorithmic problem solving.
3. Explain best-case, average-case and worst-case efficiency with a suitable example.
4. Explain the general plan of mathematical analysis of a recursive algorithm with an example.
5. Explain recurrence relations and the methods used to solve them with examples.
6. Explain time complexity, space complexity and time-space trade-off with suitable examples.